Submitted By:

Haseeb Hassan – 2025-CYS-32

Assignment:

Pacman Game Report

Submitted To:

Mr Shahmeer Nawaz

For the fulfillment of,

Object-Oriented Programming Lab

Department of Computer

Engineering University of Engineering and Technology, Lahore

Pacman Game: Complete Technical Report

1. Introduction

This report presents a detailed analysis of a Java-based Command Line Interface (CLI) Pacman game developed using Object-Oriented Programming (OOP) principles. The purpose of this project is to demonstrate how core OOP concepts such as encapsulation, modularity, and class interaction can be applied to create a simple yet functional game.

The game simulates a basic Pacman environment where the player controls a character (Pacman) inside a maze, collects food pellets, and avoids an enemy (ghost). The entire system runs in a console environment, making it lightweight and easy to understand for beginners.

2. Objectives of the Project

The main objectives of this project are:

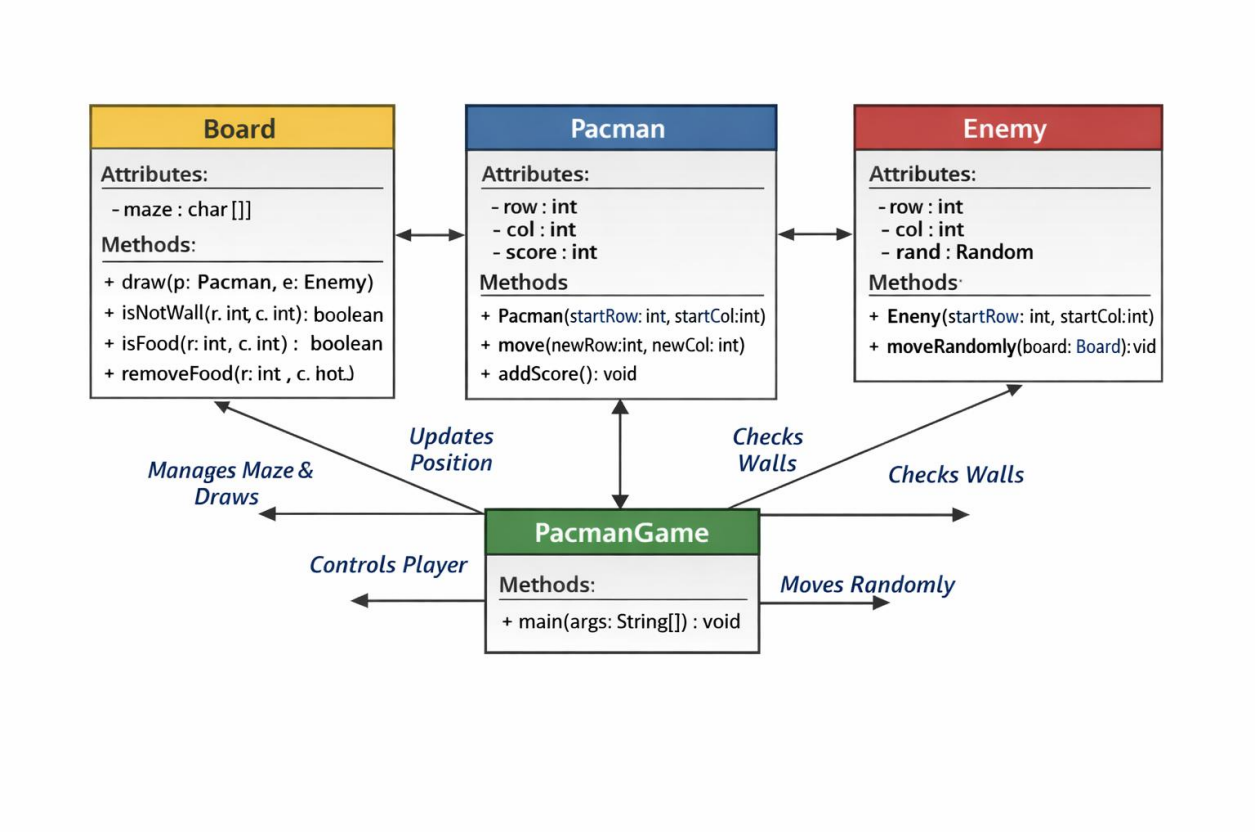
- To implement OOP concepts in Java
- To design a modular game structure
- To simulate a grid-based game environment
- To handle user input and real-time game updates
- To demonstrate interaction between multiple classes

3. System Overview

The system consists of four main classes:

1. PacmanGame (Main Class)
2. Pacman (Player Class)
3. Enemy (Ghost Class)
4. Board (Game Environment Class)

Each class has a specific responsibility, ensuring separation of concerns and better maintainability.



4. Detailed Class Description

4.1 PacmanGame Class (Main Engine)

This is the main class that controls the execution of the program. It contains the `main()` method and manages the game loop.

Responsibilities:

- Initializes game objects (Board, Pacman, Enemy)
- Takes user input using Scanner
- Controls game flow using a loop
- Updates player and enemy positions
- Checks collision conditions
- Displays output on the console

Key Features:

- Uses a `while` loop to keep the game running
- Converts user input into movement directions
- Handles game termination when collision occurs

4.2 Pacman Class (Player)

This class represents the player in the game.

Attributes:

- `row` – current row position
- `col` – current column position
- `score` – player score

Methods:

- `move(int newRow, int newCol)` → updates position
- `addScore()` → increases score when food is eaten

Functionality:

- Tracks player's movement
- Updates score dynamically

4.3 Enemy Class (Ghost)

This class represents the enemy that tries to catch Pacman.

Attributes:

- `row`, `col` – position of the enemy
- `Random rand` – used for random movement

Methods:

- `moveRandomly(Board board)` → moves enemy in random direction

Functionality:

- Generates random movement
- Ensures enemy does not pass through walls

4.4 Board Class (Game Environment)

This class defines the game grid and handles rendering.

Attributes:

- `char[][] maze` – 2D array representing the map

Symbols Used:

- # → Wall
- `.` → Food pellet
- P → Pacman
- E → Enemy

Methods:

- `draw(Pacman p, Enemy e)` → displays the board
- `isNotWall(int r, int c)` → checks valid movement
- `isFood(int r, int c)` → checks for food
- `removeFood(int r, int c)` → removes food after eating

Functionality:

- Maintains game environment
- Controls rendering and validation

5. Game Flow Explanation

The game follows a continuous loop until a collision occurs:

1. Display the board
2. Take user input (W, A, S, D)
3. Calculate new position
4. Check if move is valid
5. Update Pacman position
6. Check for food and update score
7. Move enemy randomly
8. Check collision between Pacman and Enemy
9. End game if collision occurs

6. OOP Concepts Used

Encapsulation

Each class contains its own data and methods, keeping implementation details hidden.

Abstraction

Complex logic such as movement and collision is simplified through methods.

Modularity

The program is divided into separate classes, making it easier to manage and extend.

Reusability

Classes like Board and Enemy can be reused in other similar projects.

7. Advantages of the System

- Simple and easy to understand
- Demonstrates real use of OOP concepts
- Modular and maintainable code
- Interactive CLI-based gameplay

8. Limitations

- No graphical interface (GUI)
- Enemy movement is purely random (no AI)
- Fixed board size
- Limited game features

9. Future Improvements

- Add GUI using JavaFX or Swing
- Implement intelligent enemy AI
- Add levels and difficulty
- Introduce sound effects and animations

10. Conclusion

This Pacman CLI game is a strong example of how Object-Oriented Programming can be applied in game development. Although simple, it effectively demonstrates class interaction, modular design, and real-time logic handling. With further improvements, it can be extended into a fully functional graphical game.